Response to the committee

A Lightweight Canonicalization Engine in C for Disciplined Convex Programming

William Zijie Zhang (2362162), Master's thesis, Electrical Engineering, Polytechnique Montréal

Supervisor: Prof. Antoine Lesage-Landry

October 29, 2026

To Prof. Sébastien Le Digabel (chair), Prof. Antoine Legrain (second member) and Prof. Antoine Lesage-Landry (supervisor).

Message au jury

Bonjour chers membres du jury,

Je voulais tout d'abord vous remercier pour vos commentaires sur mon mémoire. Comme vous m'avez demandé de clarifier et d'approfondir plusieurs points des résultats, j'ai pris le temps de les rendre plus complets, et j'ai aussi découvert quelques erreurs dans les affirmations que j'avais faites sur les contributions.

Premièrement, il y a maintenant quatre problèmes qui sont plus lents. J'explique la cause de chacun au chapitre 4 (§4.1), avec la part des phases principales du temps, et je donne le détail à la section 3. Pour deux d'entre eux, l'écart disparaît presque ou s'inverse avec des changements dans le « frontend » (par frontend, je fais référence à la manipulation des arbres d'expression avant le calcul des dérivées) : UnconstrainedQP passe à 16 ms contre 13 ms pour `CVXPY`, avec le même changement pour tous les backends, et OptimalAdvertising devient plus rapide (0,33 s contre 0,53 s). Par exemple, pouvoir détecter que `D @ ones` (où `D` est une matrice quelconque) équivaut à `D.sum(axis=1)`, la somme sur les colonnes, est un grand avantage. Un troisième, ConvexPlasticity, devient 2,5 fois plus rapide avec un test de densité, mais reste plus lent que `CVXPY`. Ces changements ne règlent pas tout : une partie du ralentissement vient de `SparseDiffEngine` lui-même (voir Q5).

Deuxièmement, j'ai refait toutes les mesures sur un ordinateur plus récent (MacBook M2 de 2022, avec Apple `Accelerate`), plutôt que sur l'ordinateur Intel de 2018 de la version soumise, pour que les résultats soient reproductibles sur du matériel actuel, comme celui de la plupart des utilisateurs de `CVXPY`. Pour la première canonicalisation, `diffengine` reste le plus rapide sur 21 des 25 problèmes, mais l'accélération moyenne passe de 2,8 à 2,3 fois, en partie à cause d'UnconstrainedQP (troisième point). Pour les re-résolutions paramétriques, en revanche, `diffengine` n'est plus rapide que sur trois des sept problèmes, contre cinq dans la version soumise ; sur les quatre autres, la plus grande partie d'une re-résolution est du post-traitement, après `SparseDiffEngine`. J'ai donc modifié ma conclusion et la discussion des résultats pour dire seulement qu'il y a des bénéfices, mais que la séparation ne remplace pas encore le tenseur pour les re-résolutions. Réduire ce post-traitement est maintenant le premier des travaux futurs.

Troisièmement, le problème UnconstrainedQP n'était pas réellement 100 fois plus rapide.

C'était dû à un changement de frontend que j'avais appliqué sur la branche du backend `diffengine`, mais pas sur celles des autres. En l'appliquant à tous, cet avantage disparaît. Ceci a été rectifié : aucun backend n'utilise maintenant ce changement.

À part ces trois points, qui sont les changements majeurs, j'ai aussi fait les études d'ablation et exécuté plus d'instances de benchmark, avec des mesures de signification statistique, ce qui répond à tous vos commentaires.

1 Summary of corrections

Main findings. The new measurements answer the committee's comments as follows.

- The dense blocks help only on problems with dense data. They slow down the three problems that build a dense block from data that is mostly zeros, and change little on the others.
- The dense blocks therefore do not explain the speedup, which remains almost the same without them. It comes from the engine as a whole, whose parts are built into every atom and cannot be turned off separately.
- The re-solve speed comes from the symbolic and numeric split: without it, a re-solve repeats the conversion and the symbolic pass and is 2.9 times slower.
- The speedup holds across sizes, except on three of the slower problems. Where `diffengine` runs out of memory and `CVXPY` does not, the dense blocks are usually the cause.
- The dense blocks pay off only when about half or more of the entries are nonzero.
- Each of the four slower problems has an identified cause, and three of them improve with a change made before the engine.

I have made all the modifications required by the committee and by Prof. Antoine Legrain's examiner report, as well as the minor corrections that Prof. Sébastien Le Digabel annotated and those listed in the appendix of Prof. Legrain's report. Prof. Le Digabel's corrections are not listed here; those of Prof. Legrain are in Table 2. All changes since the version sent to the committee are marked in `Document-diff.pdf`, with added text in blue and deleted text in red. Section, table and page numbers below refer to the final version. The code of the benchmarks of Chapter 4 of the thesis and of Section 3 below is in the repository `https://github.com/SparseDifferentiation/canonicalization-benchmarks`.

I wrote first drafts of these answers myself and used Claude, the AI tool declared in the thesis, to help with the results and the writing.

Table 1 lists the required modifications. "J" refers to the committee's requirements after the defence, and (i)–(vii) to the items of the synthesis of the examiner report. The defence questions are answered in Section 2; the other items are in the thesis sections listed.

Table 1: Required modifications and where they are made.

Request	Change	Where
J1a: statistical significance	30 runs per comparison; Mann–Whitney [1] per problem, Holm-adjusted [2]; Wilcoxon [3] and a bootstrap interval over the suite.	Ch. 4 introduction, p. 51; §4.1
J1b, (ii): ablation	Dense blocks turned off on all 25 problems, in time and memory; the split ablated on the re-solves (Q2).	§4.1, p. 53; Tables 8 and 9
J1c: more instances	New parameter values for each run, every problem scaled in size, and eight in density.	Ch. 4 introduction; §4.1; Appendix B
J1d: sensitivity to size and sparsity	All 25 problems at five sizes and eight at seven densities, with and without the dense blocks.	§4.1, p. 53; Appendix B, p. 73
J1e, (iii): non-DPP problems	Re-solves of two families in DPP, non-DPP and parameter-free variants, and of ConvexPlasticity (Q3).	§4.2, p. 56
J2, (i): related literature and DNLP	Matrix-free modelling, `CVXcanon`, `CVXGEN`, `Convex.jl` 0.16, `Pyomo`, block-sparse antecedents and sparse factorization; the authorship of `SparseDiffEngine` and what is specific to this work (Q6).	§2.2, §1.5, §3.1.1
J3: declaration of AI use	New front-matter page.	Front matter
(iv): repeated measurements	30 runs, each in a fresh process; tables give mean ± standard deviation.	Ch. 4 introduction; Table 4.1 (here Table 7)
(v): dimensions of the instances	Dimensions, nonzeros, density, parameter size and DPP status of every problem.	Appendix A, Tables A.1 and A.2 (here Tables 14 and 15)
(vi): time by phase	The share of each phase on the four slower problems, with the cause of each.	§4.1, p. 52; Table 10
(vii): correctness check	$\mathbf{P}$, $\mathbf{A}$, $\mathbf{c}$ and $\mathbf{b}$ are checked against the `CPP` backend to within 10^{-9} of their largest entry, at the first canonicalization and after a parameter update.	Ch. 4 introduction
Point 6: complexity	Storage of a permuted dense block against CSR, and the cost of the symbolic and numeric passes of a product against Gustavson's algorithm.	§3.3.1

The text added to the thesis for the literature, the statistics, the correctness check and the complexity is quoted below, with citations omitted.

Changes in the thesis, J2, (i): §2.1.3, `CVXcanon` and `CVXGEN`

the C++ backend `cvxcore` keeps one sparse matrix per parameter slice and loops over the slices at every operator. It began as `CVXcanon`, a C++ library written to replace the Python

matrix stuffing of early versions of `CVXPY`, and was renamed `cvxcore` in 2018. [...] `CVXPYgen` follows the earlier code generator `CVXGEN`, which generates a custom C solver for a family of small quadratic programs.

Changes in the thesis, J2, (i): §2.3.1, `Pyomo`

`Pyomo`, which also writes `.nl` files, avoids a new file per parameter change for solvers with an in-memory interface: its persistent solver interfaces keep the model inside the solver and update the changed parameters and constraints in place.

Changes in the thesis, J2, (i): §2.3.2, `Convex.jl`

`Convex.jl`, the DCP language of the Julia ecosystem, also targets MOI. Since version 0.16, it represents every affine expression as a sparse matrix and an offset over the solver variables, accumulated as the problem is converted to MOI. Like the backends of `CVXPY`, it stores the coefficients in a general-purpose sparse format.

Changes in the thesis, J2, (i): §3.2, block-sparse antecedents

Keeping dense blocks inside a sparse matrix is not a new idea. Finite-element codes assemble dense element matrices into a sparse global matrix, and nonlinear least-squares libraries such as `Ceres` and `g2o` store the block-sparse Jacobians of bundle adjustment as dense blocks. In those codes, the position of each block follows from the structure of the problem, one element or one residual at a time. In our representations, the symbolic pass derives it from the expression tree.

Changes in the thesis, J2, (i): §1.1, the symbolic and numeric split

This separation between structure and values goes back to sparse direct solvers, which analyse the sparsity pattern of a matrix once and repeat the numeric factorization for new values, and to Gustavson's sparse product.

Changes in the thesis, J1a, J1c, (iv): Chapter 4, Statistics

Each measurement is made in 30 runs, each in a fresh process (10 for the three points of Appendix B that take over 30 s), and on a parametric problem each run uses new parameter values. The tables report the mean and the standard deviation of the runs, and the ratios are ratios of medians. On each problem, the runs of `diffengine` and of the other backend are compared with a two-sided Mann–Whitney U test, with the p-values adjusted over the problems by the Holm–Bonferroni method. Over the suite, we test whether the geometric mean of the ratios differs from 1 with a Wilcoxon signed-rank test on their logarithms, and give a bootstrap 95 % confidence interval for it.

Changes in the thesis, (vii): Chapter 4, Setup

Before any timing, we check that `diffengine` returns the same problem data as `CPP`, to within 10^{-9} of the largest entry, at the first canonicalization and after a parameter update. [Footnote:] Except for one NaN entry that comes from the data of OptimalAdvertising itself.

Changes in the thesis, Point 6: §3.3.1, complexity

A permuted dense matrix with an $m_0 \times n_0$ block stores $m_0 n_0$ values and $m_0 + n_0$ indices, against $m_0 n_0$ values and $m_0 n_0 + m + 1$ indices for the same block in CSR form, where m is the number of rows of the whole matrix. For (3.2), [...] the symbolic pass costs $O(m_A + m_B)$ and the numeric pass performs $n_B \, s \, n_A$ multiply-adds in one `dgemm`, whereas Gustavson's algorithm performs the same multiply-adds after a symbolic pass of $O(n_B \, s \, n_A)$.

Table 2: Minor corrections of Prof. Legrain's report.

Correction	Change
Résumé: « librairie »	« bibliothèque logicielle ».
§4.3: « 24 of the 25 » contradicts « 21 of 25 »	« canonicalizes faster than `cvxcore` on 23 of the 25 problems » (§4.3); the count changed with the new measurements.
Harmonize the main contribution (§1.5, Ch. 3, §3.5)	The same order everywhere, the matrix classes first and the split second, with no "main contribution".
Symbolic/numeric split: cite sparse factorization first	Davis and Gustavson cited before the NLP interfaces (quoted above).
Figure 3.5 relies on colour alone	Each cell and the legend carry a letter (a)–(d) (Figure 3.5).
References: « p. » for page ranges	Ranges now print as « pp. ».
References to Eigen, CppAD and MOSEK incomplete	URLs added; Eigen version 3.4.0; MOSEK documentation.
Status of the merged pull request	Merged on 9 September 2026, opt-in, parameter-free problems only; the parametric path is under review (§5.2).
The author contributed to `SCIPY` and `COO`	Stated in §5.2.
Delineation from DNLP (§1.5, §3.1)	Q6.
Table 2.2 and Pyomo	A note under the table and the paragraph quoted above.

2 Replies to the defence questions

Q1. *Why materialize dense blocks rather than leave them as operators, as in the matrix-free approach of Diamond and Boyd [4]? When is each approach preferable?*

Because the solvers that `CVXPY` calls need the matrices. Its default solvers are `Clarabel`, an interior-point method, and `OSQP` and `SCS`, which are first-order methods; commercial solvers such as `MOSEK` and `Gurobi` are interior-point or simplex methods. All of them factorize a matrix built from $\mathbf{A}$ and $\mathbf{P}$: an interior-point method factorizes a KKT system at every iteration, and `OSQP` and `SCS`, in their default direct mode, factorize their KKT matrix and reuse the factorization across iterations, refactorizing only when a step-size parameter changes. So

every one of them needs $\mathbf{A}$ and $\mathbf{P}$ explicitly, whatever its algorithm. Matrix-free modelling pays off for very large problems built from fast transforms, solved by first-order methods that only need products with $\mathbf{A}$ and $\mathbf{A}^\top$, such as the indirect mode of `SCS`. A permuted dense block lies between the two: it is a materialized matrix, but it keeps the structure of the operator, namely its row and column selectors and its dense block.

Changes in the thesis, §2.2, p. 25

In convex optimization, Diamond and Boyd take the matrix-free view. Instead of forming $\mathbf{A}$, they represent the problem as a graph of linear operators, available only through their forward and adjoint products, which suits first-order solvers. The solvers that `CVXPY` calls by default instead factorize a matrix built from $\mathbf{A}$ and $\mathbf{P}$. The matrix-free approach pays off on very large problems built from fast transforms, such as convolutions and Fourier transforms, and it needs solvers that accept operators instead of matrices. The permuted dense matrices of §3.2 sit between the two: they form the matrix that the solver factorizes, but keep the structure of the operator, its row and column selectors and its dense block.

Q2. *If the permuted dense representations are disabled, what fraction of the speedup remains?*

Over the suite, the speedup of `diffengine` changes little without the dense blocks. The 2.8× of the submitted version is now 2.29×, in part because UnconstrainedQP no longer benefits from a rewrite that the other backends did not get (see the message to the committee). A switch of our `CVXPY` branch stores every constant matrix in the sparse format. With sparse matrices alone, `diffengine` is still 2.15 times faster than the best `CVXPY` backend over the suite, close to the 2.29 times with the dense blocks ($p = 0.07$). The remaining 2.15× thus comes from the rest of the engine, which our switches cannot divide further: its implementation in C, the Jacobian it computes at each node and the global layout are built into every atom. Per problem, the dense blocks gain up to a factor of 3 on problems with dense data, and lose up to a factor of 46 on CVaR, ConvexPlasticity and UnconstrainedQP, which build a dense block from data that is mostly zeros (Tables 7 and 8). Without them, `diffengine` also uses 0.83 times the peak memory in geometric mean (Table 9).

The split between the symbolic and numeric passes can be turned off on re-solves only; without it, a re-solve is 2.9 times slower, significantly on all seven problems (Table 3). We conclude that the first-canonicalization speedup comes from the engine as a whole rather than from the dense blocks, which pay off only on data whose entries are mostly nonzero, and that the re-solve speed comes from the split.

Table 3: Not in the thesis. The ablations in summary: geometric mean of the ratio of each variant to its reference, its bootstrap 95 % confidence interval, the Wilcoxon p-value over the problems, and the number of problems with a significant difference (Mann–Whitney, Holm-adjusted). The two that are not significant, ParametrizedQP on the first canonicalization and SimpleScalarParametrizedLP on the re-solve, have no dense constant data, so turning the dense blocks off barely changes their work.

Variant	Reference	Ratio	95 % CI	p	Significant
First canonicalization, 25 problems					
no dense blocks	`diffengine`	1.06	0.71–1.45	0.067	24/25
`diffengine`	best `CVXPY`	0.44	0.29–0.70	0.003	25/25
no dense blocks	best `CVXPY`	0.46	0.36–0.61	3.2×10^{-5}	25/25
Re-solve, 7 DPP problems					
no dense blocks	`diffengine`	1.01	0.98–1.05	0.938	6/7
no split	`diffengine`	2.86	1.73–4.91	0.016	7/7
neither	`diffengine`	3.68	2.19–6.20	0.016	7/7

Q3. *How does re-solving behave on a non-DPP parametric problem, for instance with a product of two parameters or a parameter in a non-affine position?*

On the two families written with a product of two parameters, which DPP forbids, `CVXPY` has to canonicalize the problem again at every solve, while `diffengine` keeps its symbolic pass and reruns only the numeric pass. It re-solves as fast as on the DPP variant, 13 to 14× faster than `CVXPY` on `lasso` and `factor_cov`, significantly on both (Table 4). ConvexPlasticity, the suite's only natural non-DPP parametric problem, breaks DPP for the second reason: its parameters lie inside the argument of a quadratic form, which `CVXPY` requires to be parameter-free for a solver that accepts a quadratic objective. It is the exception: it re-solves in 207 ms against 42 ms for `CVXPY`, because 88 % of the re-solve assembles a dense 3,000 × 3,000 block of $\mathbf{P}$ built from a diagonal. Without the dense blocks, it re-solves in 2.0 ms, 21× faster than `CVXPY`. The thesis no longer claims that the split replaces the tensor (Q4).

The thesis states the results of Table 4 in §4.2 without the table:

Changes in the thesis, §4.2, p. 56

The numeric pass needs a fixed expression tree, not the DPP rules. We write the `lasso` and FactorCovarianceModel with one parameter as the product of two, which DPP forbids. `CVXPY` then canonicalizes again at every solve, while `diffengine` reruns only its numeric pass, 13 to 14 times faster. The exception is ConvexPlasticity, where the re-solve is spent assembling a dense block of $\mathbf{P}$.

Changes in the thesis, §4.3, p. 57

Caching canonicalization does not necessarily require the DPP ruleset. [...] the symbolic and numeric split re-solves within a factor of four of the cached tensor on every DPP problem. [...] it re-solves problems that are not DPP, on which `CVXPY` has to canonicalize again, 13 to 14

Table 4: Not in the thesis. Re-solve time in milliseconds, mean over 30 runs; every standard deviation is below 1 % of the mean. `factor_cov` is FactorCovarianceModel with n assets. The non-DPP variant writes λ, or the risk aversion γ, as the product of two parameters, and the parameter-free variant makes it a constant. `CVXPY` applies the tensor on the DPP variant and canonicalizes again on the others; the cached column of `diffengine` runs the numeric pass only, and the rebuild column the symbolic pass as well. The ratio is the median of `diffengine`, cached when it can, over that of `CVXPY`, * when significant.

			`diffengine`		
Family	Variant	`CVXPY`	cached	rebuild	Ratio
`lasso`, $n = 1{,}000$	DPP	6.4	23.3	32.7	3.64*
	non-DPP	315	23.3	32.8	0.07*
	parameter-free	338	—	49.0	0.14*
`factor_cov`, $n = 5{,}000$	DPP	1.2	3.6	6.9	2.86*
	non-DPP	45.8	3.6	7.0	0.08*
	parameter-free	47.1	—	11.0	0.23*

times faster.

Q4. *Why not re-evaluate only the subtrees that depend on the updated parameters?*

The engine now does this: after a parameter update, the numeric pass re-evaluates only the subtrees that contain a parameter, and keeps the cached values of the others. All re-solve times in the thesis use it.

The prediction of the examiner report, that this would make `diffengine` at least as fast as the tensor in every case, does not hold. On the seven DPP problems (Table 5), `diffengine` has the fastest first canonicalization on all seven, 2.5 to 6.6× faster than the fastest tensor build. The cached tensor re-solves faster on four of them, by 1.10 to 3.60×, and recovers its slower first canonicalization only after 18 to 124 re-solves. On those four, the numeric pass takes only 4 to 24 % of a re-solve; the rest is post-processing after the engine, the conversion to the solver's format. We conclude that the split does not replace the tensor on DPP problems that are re-solved many times, but comes within a factor of four of it without the DPP rules. Making the post-processing cheaper is now the first item of future work.

Changes in the thesis, §3.4.1, p. 47

The numeric pass therefore skips an affine tree after the first evaluation, until the next parameter update. After an update, it re-evaluates only the subtrees that contain a parameter, and keeps the cached values of the others.

Table 5: Table 4.2 of the thesis (p. 56): first canonicalization time in seconds and re-solve time in milliseconds on the seven DPP parametric problems, mean over the runs, with the standard deviation of the re-solves. A dash denotes a tensor that exceeded the memory limit or the time limit. Bold denotes the fastest first canonicalization and the faster re-solve. The tensor column is the fastest of the three `CVXPY` backends, which agree within 5 %. The ratio is the median re-solve time of `diffengine` over that of the tensor, marked with * when significant.

	first canonicalization (s)				re-solve (ms)		
Benchmark	`CPP`	`SCIPY`	`COO`	`diffengine`	tensor	`diffengine`	Ratio
ParamSmallMatrixStuffing	2.02	7.30	0.083	**0.033**	1.84 ± 0.01	**0.438 ± 0.007**	0.24*
ParamConeMatrixStuffing	0.74	1.85	0.055	**0.022**	0.958 ± 0.005	**0.265 ± 0.004**	0.27*
SimpleFullyParametrizedLP	—	—	0.57	**0.12**	46.7 ± 0.1	**34.6 ± 0.8**	0.73*
SimpleScalarParametrizedLP	1.33	1.17	1.19	**0.23**	**62.8 ± 0.3**	70.4 ± 5.9	1.10*
ParametrizedQP	—	—	2.63	**0.76**	**299 ± 8**	349 ± 16	1.16*
SVMWithL1Regularization	2.80	2.57	2.83	**0.39**	**38.3 ± 0.1**	125 ± 2	3.26*
FactorCovarianceModel	1.20	1.14	1.19	**0.18**	**20.5 ± 0.1**	74.2 ± 0.8	3.60*

Q5. *When does the global layout become a handicap relative to `CVXPY`'s local layout, in particular for memory?*

When the problem has many variables and many nodes. In the global layout, each node keeps, besides its Jacobian, integer work arrays of length n, the total number of variables, whatever variables its own subtree contains. Their memory is therefore $O(\text{nodes} \times n)$ integers. In `CVXPY`'s local layout, each node stores blocks only for the variables it depends on, so the same overhead is proportional to the sum, over the nodes, of the number of variables each one depends on. The two coincide when every node depends on every variable, and the local layout wins when most nodes depend on a few variables out of many.

OptimalAdvertising shows it. Each of its 250 minimum terms depends on one row of a 250 × 1,000 variable, but has a Jacobian over all 250,000 variables. Without the dense blocks, its peak memory is 945 MB, against 173 to 197 MB for the `CVXPY` backends. With them it reaches 6.1 GB, because each dense block of the aggregate constraints $\mathbf{D}^\top \mathbf{1} \leq \mathbf{t}$ and $\mathbf{D1} \geq \mathbf{c}$ also keeps one integer per variable of the problem: the same cost, paid per block rather than per node. Rewriting these products with a vector of ones as sums avoids the dense blocks: OptimalAdvertising then canonicalizes in 0.33 s instead of 0.91 s, faster than the 0.53 s of `CVXPY`, with 1.4 GB instead of 6.1 GB.

Q6. *What part of `SparseDiffEngine` pre-existed the thesis or was developed for DNLP, and what part is specific to this work?*

`SparseDiffEngine` is joint work with Daniel Cederberg. It was first built as the differentiation engine of the `DNLP` extension of `CVXPY` [5], which differentiates nonlinear atoms at every iterate of the solver. The version sent to the committee should have said so clearly; it now does. This thesis uses the engine for a different purpose, canonicalization: it evaluates the affine atoms and the quadratic form at the origin to obtain the problem data. What is specific to the thesis is the interface to `CVXPY`, namely the `diffengine` backend and its

re-solve path for parametric problems, and the benchmarks of Chapter 4.

Changes in the thesis, §1.5, p. 16

`SparseDiffEngine` is joint work with Daniel Cederberg. It was first built as the differentiation engine of the `DNLP` extension of `CVXPY`, where it differentiates nonlinear atoms such as `exp`, `log`, and `entr` at every iterate of the solver. This thesis builds on the observation that canonicalization is a special case of the same computation: once a problem is canonicalized, only affine atoms and quadratic forms remain, and their derivatives are the problem data.

Changes in the thesis, §3.1.1, p. 37

What is specific to this thesis is the interface to `CVXPY` of §3.4, namely the `diffengine` backend and its re-solve path for parametric problems, and the benchmarks of Chapter 4.

Q7. *What is the strategy for BLAS and binary distribution of* ***`sparsediffpy`****?*

`cvxcore` is a C++ extension that lives inside the `CVXPY` repository, so `CVXPY` has to compile and ship it. `sparsediffpy` is a separate package that `CVXPY` only depends on. Its compiled wheels for Linux, macOS and Windows are published on `PyPI`; they link Apple's `Accelerate` on macOS and bundle `OpenBLAS` elsewhere, so users install nothing beyond the package. The binary does not go away, but it is built and released independently of `CVXPY`, like its other compiled dependencies, which is what the removal of `cvxcore` aims at.

3 Results: the new tables and what they show

The thesis adds the two tables of Appendix A and the two of Appendix B, and redoes two tables of Chapter 4 (Table 6). To keep the thesis short, six other tables are not in it: Tables 3 and 4 in the replies above, and the four below. The thesis states their results in a sentence or two, and I give them here in full. After each table I say what I conclude from it. All are measured as in Chapter 4, with the code at `https://github.com/SparseDifferentiation/canonicalization-benchmarks`.

Table 6: Tables added to the thesis, or revised.

Item	Content	Request
Table 4.1	Revised: first canonicalization of the 25 problems, now mean ± standard deviation over 30 runs with significance, and the ratio without the dense blocks, p. 55; here Table 7	J1a, J1b, (ii), (iv)
Table 4.2	Revised: first canonicalization and re-solve of the DPP problems, with standard deviations and significance, p. 56; here Table 5	J1a, (iv), Q4
Table A.1	The 25 benchmark problems, p. 70; here Table 14	(v)

Item	Content	Request
Table A.2	Dimensions of the benchmark problems, p. 72; here Table 15	(v)
Table B.1	Ratio at seven densities, with and without the dense blocks, p. 73; here Table 11	J1c, J1d
Table B.2	Ratio to the best `CVXPY` backend at five sizes, p. 74; here Table 12	J1c, J1d

3.1 First canonicalization (J1a, (iv))

Table 7: Table 4.1 of the thesis (p. 55): `ignore_dpp` canonicalization time in milliseconds on all 25 problems, mean ± standard deviation over the runs. Bold denotes the fastest of the four backends on each problem. The ratio is the median time of `diffengine` over that of the best of `CVXPY`'s three backends, marked with * when significant. The last column is the same ratio with the dense blocks turned off (§4.1); this ratio is also significant on every problem. The final row is the geometric mean of the ratios.

					Ratio	
Benchmark	`CPP`	`SCIPY`	`COO`	`diffengine`	dense	no dense
SVMWithL1Regularization	2,786 ± 8	2,634 ± 8	2,742 ± 15	**387 ± 1**	0.15*	0.28
FactorCovarianceModel	1,223 ± 5	1,164 ± 3	1,150 ± 4	**177 ± 3**	0.15*	0.32
Yitzhaki	938 ± 1	982 ± 2	1,006 ± 3	**145 ± 1**	0.15*	0.28
SimpleQP	2,891 ± 5	2,944 ± 6	2,929 ± 6	**453 ± 1**	0.16*	0.32
HuberRegression	2,770 ± 6	2,850 ± 7	2,796 ± 7	**495 ± 8**	0.18*	0.37
LeastSquares	2,410 ± 9	2,912 ± 7	2,939 ± 5	**435 ± 2**	0.18*	0.38
SimpleLP	6,898 ± 38	6,303 ± 17	5,962 ± 14	**1,121 ± 3**	0.19*	0.25
SimpleScalarParametrizedLP	1,260 ± 3	1,151 ± 2	1,087 ± 3	**230 ± 1**	0.21*	0.21
SimpleFullyParametrizedLP	611 ± 1	555 ± 1	523 ± 1	**115 ± 0.5**	0.22*	0.22
SlowPruning	1,792 ± 4	2,185 ± 4	2,147 ± 4	**408 ± 1**	0.23*	0.23
ParametrizedQP	2,974 ± 29	3,030 ± 7	2,990 ± 10	**755 ± 46**	0.25*	0.25
Cajas	3,885 ± 12	689 ± 5	693 ± 5	**205 ± 1**	0.30*	0.30
ConeMatrixStuffing	2,517 ± 10	2,152 ± 8	3,537 ± 8	**642 ± 4**	0.30*	0.90
SmallMatrixStuffing	1,683 ± 13	1,822 ± 8	2,508 ± 7	**502 ± 2**	0.30*	0.81
TvInpainting	919 ± 1	913 ± 2	815 ± 2	**257 ± 1**	0.32*	0.31
ParamSmallMatrixStuffing	63.3 ± 0.2	127 ± 1	113 ± 1	**32.7 ± 0.1**	0.52*	0.64
ParamConeMatrixStuffing	42.8 ± 0.2	71.9 ± 0.3	88.7 ± 0.4	**22.4 ± 0.4**	0.52*	0.64
CVaR	7,196 ± 15	6,503 ± 20	6,512 ± 23	**3,421 ± 9**	0.53*	0.37
SemidefiniteProgramming	1,161 ± 3	632 ± 2	997 ± 4	**405 ± 3**	0.64*	0.64
QuantumHilbertMatrix	1,062 ± 2	805 ± 4	788 ± 2	**653 ± 3**	0.83*	0.85
Murray	1,276 ± 18	1,229 ± 7	1,224 ± 7	**1,181 ± 1**	0.96*	0.96
SDPSegfault1132	8,000 ± 11	**1,663 ± 148**	7,298 ± 110	2,147 ± 6	1.32*	2.12
OptimalAdvertising	2,573 ± 3	598 ± 8	**534 ± 1**	901 ± 4	1.68*	1.56
ConvexPlasticity	78.2 ± 0.7	72.0 ± 0.7	**69.8 ± 1.2**	507 ± 1	7.31*	1.39
UnconstrainedQP	**1,955 ± 3**	1,980 ± 5	2,439 ± 2	22,937 ± 21	11.73*	0.26
Geometric mean					**0.44**	**0.46**

Table 7 is the main result. `diffengine` is the fastest backend on 21 of the 25 problems,

significantly on all 21, and its ratio to the best `CVXPY` backend is 0.44 in geometric mean. The table is sorted by that ratio, so the four slower problems are the last four rows; I explain each one in §3.4. Comparing the two ratio columns shows where the dense blocks matter. On the problems with dense data, from SVMWithL1Regularization to LeastSquares and on the two MatrixStuffing problems, the ratio is two to three times higher without them. On CVaR, ConvexPlasticity and UnconstrainedQP it is lower. On most of the other problems the two columns agree, because turning the dense blocks off changes little of their work. `diffengine` is significantly faster than the best `CVXPY` backend on 21 of the 25 problems, and 2.3 times faster in geometric mean.

3.2 Dense blocks (J1b, (ii), Q2)

Table 8 groups the problems by the effect of the dense blocks, and the groups follow the data.

Dense blocks help. On these twelve problems, the constant data is mostly nonzero, and turning the dense blocks off makes `diffengine` 1.2 to 3 times slower. On ConeMatrixStuffing and SmallMatrixStuffing, this data is the dense matrix of the objective $\mathbf{1}^\top \mathbf{A}\mathbf{x}$, which is why the density of their constraint matrix in Table 15 is low.

Little effect. On these ten problems, turning the dense blocks off changes the time by at most 7 %, and by at most 3 % on all but OptimalAdvertising. Murray, for instance, has a dense array of differences, but at a density below 5 %, where `diffengine` already stores a constant factor in the sparse format. These small differences are significant only because the runs vary so little.

Dense blocks hurt. On these three problems, a dense block is built from data that is mostly zeros, and turning the dense blocks off makes `diffengine` 1.4 to 46 times faster. The data is the scenario matrix of CVaR, a dense array with one nonzero in four; the diagonal of ConvexPlasticity, given as a dense array; and the chain of products of UnconstrainedQP, whose dense Jacobian comes from a sparse factor (§3.4).

The dense blocks speed up the problems with dense data, slow down the three that build them from mostly zeros, and change little on the rest.

Table 8: Not in the thesis. First canonicalization of `diffengine` with and without the dense blocks, in milliseconds. The ratio is the median time without the dense blocks over that with them, * when significant. The ratios to the best `CVXPY` backend, with and without the dense blocks, are the last two columns of Table 7.

Benchmark	`diffengine`	no dense blocks	Ratio
Dense blocks help			
SVMWithL1Regularization	387 ± 1	745 ± 5	1.92*
FactorCovarianceModel	177 ± 3	366 ± 6	2.06*
Yitzhaki	145 ± 1	263 ± 1	1.82*
SimpleQP	453 ± 1	914 ± 6	2.01*
HuberRegression	495 ± 8	1,015 ± 3	2.06*
LeastSquares	435 ± 2	906 ± 2	2.08*
SimpleLP	1,121 ± 3	1,485 ± 5	1.32*
ConeMatrixStuffing	642 ± 4	1,941 ± 7	3.03*
SmallMatrixStuffing	502 ± 2	1,369 ± 6	2.73*
ParamSmallMatrixStuffing	32.7 ± 0.1	40.4 ± 0.3	1.23*
ParamConeMatrixStuffing	22.4 ± 0.4	27.4 ± 0.7	1.22*
SDPSegfault1132	2,147 ± 6	3,466 ± 15	1.61*
Little effect			
SimpleScalarParametrizedLP	230 ± 1	225 ± 0.3	0.98*
SimpleFullyParametrizedLP	115 ± 0.5	115 ± 0.4	1.00*
SlowPruning	408 ± 1	417 ± 1	1.02*
ParametrizedQP	755 ± 46	746 ± 34	1.00
Cajas	205 ± 1	209 ± 1	1.02*
TvInpainting	257 ± 1	250 ± 1	0.97*
SemidefiniteProgramming	405 ± 3	406 ± 1	1.00*
QuantumHilbertMatrix	653 ± 3	671 ± 3	1.03*
Murray	1,181 ± 1	1,179 ± 2	1.00*
OptimalAdvertising	901 ± 4	834 ± 1	0.93*
Dense blocks hurt			
CVaR	3,421 ± 9	2,419 ± 36	0.71*
ConvexPlasticity	507 ± 1	97.1 ± 3.2	0.19*
UnconstrainedQP	22,937 ± 21	500 ± 2	0.02*
Geometric mean			**1.06**

3.3 Peak memory (J1b, (ii), Q5)

Table 9: Not in the thesis. Growth of the peak resident memory during the first canonicalization, in megabytes. Bold denotes the least of the four backends. The last column is `diffengine` without the dense blocks.

Benchmark	`CPP`	`SCIPY`	`COO`	`diffengine`	no dense blocks
SVMWithL1Regularization	**1,976**	2,041	2,904	2,034	2,417
FactorCovarianceModel	1,129	1,047	1,334	**760**	999
Yitzhaki	449	**437**	504	476	572
SimpleQP	2,194	2,355	3,236	**1,655**	2,204
HuberRegression	2,223	2,246	3,121	**1,413**	1,866
LeastSquares	2,142	2,327	3,021	**1,459**	1,893
SimpleLP	5,353	4,961	5,027	**4,792**	5,057
SimpleScalarParametrizedLP	1,088	1,024	1,068	**1,009**	1,009
SimpleFullyParametrizedLP	517	494	526	**492**	492
SlowPruning	**1,352**	1,425	1,828	1,797	1,120
ParametrizedQP	2,035	2,174	2,998	**1,775**	1,775
Cajas	1,193	**298**	320	341	342
ConeMatrixStuffing	2,010	964	3,730	**478**	1,623
SmallMatrixStuffing	1,189	634	2,405	**322**	1,117
TvInpainting	**705**	858	994	1,048	1,048
ParamSmallMatrixStuffing	17.4	12.4	21.9	**10.6**	10.5
ParamConeMatrixStuffing	9.8	8.2	12.2	**5.9**	5.8
CVaR	4,022	**2,804**	3,587	10,654	2,115
SemidefiniteProgramming	1,317	**350**	640	1,138	1,138
QuantumHilbertMatrix	222	**218**	288	1,194	1,194
Murray	2,849	**220**	305	306	307
SDPSegfault1132	7,536	**3,460**	15,628	10,413	12,885
OptimalAdvertising	**173**	182	197	6,066	945
ConvexPlasticity	82.5	82.5	**82.3**	1,329	25.5
UnconstrainedQP	1,427	**1,058**	2,105	1,754	644

Without the dense blocks, the peak memory falls most on the problems where the dense blocks also cost time: from 10.7 to 2.1 GB on CVaR, from 6.1 to 0.95 GB on OptimalAdvertising, from 1.3 GB to 26 MB on ConvexPlasticity and from 1.75 to 0.64 GB on UnconstrainedQP. It rises on the problems with dense data, where a dense block needs no index per entry: by 19 to 33 % from SVMWithL1Regularization to LeastSquares, and by a factor of about 3.4 on the two MatrixStuffing problems. In geometric mean, `diffengine` uses 0.83 times its memory with the dense blocks. On OptimalAdvertising, `diffengine` still uses five times the memory of `CVXPY` without the dense blocks, which is the cost of the global layout (Q5).

3.4 The four slower problems ((vi), Q5)

Table 10 gives the share of each phase of the first canonicalization of `diffengine` on the four problems on which it is slower. On a typical problem, the time is spread over the phases; on each of the four, one or two phases take almost all of it, and the paragraphs below say why. Three of the four are addressed by changes to `CVXPY`, which the measurements of the thesis do not include; each fix is measured alone.

Table 10: Not in the thesis. Phases of the first canonicalization of `diffengine` on the four slower problems, in percent of its time, and the median share over the 25 problems, mean over the 30 runs. Each phase is timed by a wall-clock timer in our `CVXPY` branch: the conversion of the `CVXPY` expressions into the trees of the engine, the symbolic pass, the numeric pass, the assembly of $\mathbf{A}$ and $\mathbf{P}$ in CSC format, and the formatting of the rows by cone. The front end is the rest of the time, the reductions of `CVXPY` before the engine.

		share of the time (%)					
Benchmark	Time (s)	Front end	Convert	Symbolic	Numeric	Assemble	Format
UnconstrainedQP	22.94	0	0	2	98	0	0
ConvexPlasticity	0.51	6	2	47	6	38	0
OptimalAdvertising	0.90	4	1	89	4	1	1
SDPSegfault1132	2.15	0	0	49	47	3	0
Median, all 25	—	4	9	19	14	12	12

UnconstrainedQP. The problem minimizes $\|\mathbf{F}^{\mathsf{H}}\mathbf{E}(\mathbf{z})\mathbf{F}\mathbf{x}-\mathbf{y}\|_2$, with $\mathbf{E}(\mathbf{z}) = \mathbf{I}_{14}\otimes\operatorname{diag}(\mathbf{z})$, over $\mathbf{z}\in\mathbb{C}^{18}$, where $\mathbf{F}$ is the 252×252 discrete Fourier transform matrix. Evaluated from the left, the product $(\mathbf{F}^{\mathsf{H}}\mathbf{E})\mathbf{F}$ has a dense Jacobian of $63{,}504\times36$ entries, which the numeric pass fills one entry at a time. Evaluated from $\mathbf{x}$, as the reassociation of matrix products excluded from the thesis measurements does, every intermediate is a vector, and the canonicalization takes 16 milliseconds instead of 22.9 seconds, with a peak memory of 53 megabytes instead of 1.75 gigabytes.

OptimalAdvertising. The problem maximizes $\sum_{i=1}^{250}\min(r_i\mathbf{p}_i^\top\mathbf{d}_i, b_i)$ subject to $\mathbf{D}\geq\mathbf{0}$, $\mathbf{D}^\top\mathbf{1}\leq\mathbf{t}$ and $\mathbf{D}\mathbf{1}\geq\mathbf{c}$, over $\mathbf{D}\in\mathbb{R}^{250\times1{,}000}$ with rows $\mathbf{d}_i^\top$. Each minimum term depends on one row of $\mathbf{D}$, but its Jacobian spans all 250,000 variables, and the symbolic pass visits every column of it, so its cost grows with the number of terms times the number of variables. The two aggregate constraints are written as products with a vector of ones, which become dense blocks. A change to `CVXPY` lowers them to sums, which reduces the canonicalization from 0.91 to 0.33 seconds, below the 0.53 seconds of the fastest `CVXPY` backend, and the peak memory from 6.1 to 1.4 gigabytes.

SDPSegfault1132. Three dense blocks of 49.5 million entries cancel in a sum with 1.01 million nonzeros, and `diffengine` carries the cancelled entries through both passes.

ConvexPlasticity. The diagonal $h\mathbf{I}$ of the quadratic objective is given as a dense array and becomes a dense 3,000 × 3,000 block of $\mathbf{P}$. A change to `CVXPY` stores $\mathbf{P}$ sparse below 5 % nonzeros, as is already done for a constant factor of a matrix product, which reduces the canonicalization from 509 to 203 milliseconds and the peak memory from 1.33 to 0.59 gigabytes. The problem remains 2.9 times slower than on the fastest `CVXPY` backend, and most of the remaining time is in the symbolic pass.

On each slower problem, one or two phases take almost all of the time, and three of the four improve with a change made before the data reaches the engine.

3.5 Density and size (J1c, J1d)

Table 11: Table B.1 of the thesis (p. 73): ratio of the median canonicalization time of `diffengine` to that of the best `CVXPY` backend as the density ρ of the data matrix falls, with and without the dense blocks. Below $\rho = 0.05$, `diffengine` stores a constant factor in the sparse format. Every difference is significant.

	density ρ						
Benchmark	1	0.5	0.2	0.1	0.05	0.02	0.01
with dense blocks							
SVMWithL1Regularization	0.15	0.33	0.64	0.98	1.47	0.63	0.72
FactorCovarianceModel	0.15	0.33	0.57	0.77	1.01	0.60	0.65
Yitzhaki	0.15	0.29	0.53	0.75	0.99	0.67	0.73
SimpleQP	0.16	0.30	0.47	0.56	0.63	0.40	0.41
HuberRegression	0.18	0.34	0.67	1.03	1.78	1.74	2.54
LeastSquares	0.18	0.34	0.61	0.83	1.20	0.67	0.81
SemidefiniteProgramming	0.64	0.67	1.02	1.28	1.46	1.60	1.66
`lasso`	0.15	0.33	0.59	0.84	0.58	0.75	0.86
without dense blocks							
SVMWithL1Regularization	0.28	0.32	0.36	0.41	0.48	0.62	0.71
FactorCovarianceModel	0.32	0.35	0.39	0.44	0.52	0.61	0.66
Yitzhaki	0.28	0.30	0.36	0.46	0.56	0.66	0.71
SimpleQP	0.32	0.34	0.39	0.44	0.47	0.51	0.52
HuberRegression	0.37	0.36	0.46	0.62	0.96	1.72	2.52
LeastSquares	0.38	0.36	0.40	0.43	0.53	0.66	0.79
SemidefiniteProgramming	0.64	0.68	1.02	1.28	1.46	1.61	1.67
`lasso`	0.31	0.34	0.37	0.45	0.54	0.69	0.79

As the density of the data falls (Table 11), the ratio with the dense blocks rises until about $\rho = 0.05$ on the problems that use them, because a dense block keeps storing and multiplying every entry while the `CVXPY` backends do less work. Below $\rho = 0.05$, `diffengine` stores the constant in the sparse format, and the ratio drops to about that without the dense blocks. Without them, the ratio rises slowly as the density falls. The two variants are about equal at $\rho = 0.5$, and the variant without them is faster from $\rho = 0.2$ to 0.05. The dense blocks pay off only when about half or more of the entries are nonzero.

As each problem is scaled from $s = 1/4$ to $s = 4$ (Table 12), the ratio stays within 0.15

Table 12: Table B.2 of the thesis (p. 74): ratio of the median canonicalization time of `diffengine` to that of the best `CVXPY` backend as the size of each problem is scaled by s, in the order of Table 4.1. "cap" denotes a size at which `diffengine` exceeded the 20 GB memory limit while a `CVXPY` backend finished, "all cap" one at which every backend exceeded it, and a dash a size the problem does not have. Every difference is significant.

	size factor s				
Benchmark	1/4	1/2	1	2	4
SVMWithL1Regularization	0.16	0.16	0.15	0.15	0.16
FactorCovarianceModel	0.17	0.16	0.15	0.18	0.22
Yitzhaki	0.14	0.16	0.15	0.14	0.14
SimpleQP	0.15	0.15	0.16	0.16	0.16
HuberRegression	0.16	0.19	0.18	0.17	0.17
LeastSquares	0.16	0.16	0.18	0.17	0.18
SimpleLP	0.20	0.20	0.19	0.18	all cap
SimpleScalarParametrizedLP	0.22	0.22	0.21	0.20	0.19
SimpleFullyParametrizedLP	0.24	0.22	0.22	0.21	0.20
SlowPruning	0.23	0.22	0.23	0.22	0.21
ParametrizedQP	0.17	0.23	0.25	0.23	0.22
Cajas	0.29	0.28	0.30	0.26	0.26
ConeMatrixStuffing	0.31	0.31	0.30	0.29	0.31
SmallMatrixStuffing	0.42	0.30	0.30	0.27	0.27
TvInpainting	0.33	0.33	0.32	0.31	0.29
ParamSmallMatrixStuffing	0.52	0.52	0.52	0.51	0.43
ParamConeMatrixStuffing	0.51	0.52	0.52	0.52	0.52
CVaR	0.59	0.56	0.53	cap	all cap
SemidefiniteProgramming	0.58	0.61	0.64	0.64	0.68
QuantumHilbertMatrix	—	1.17	0.83	all cap	—
Murray	1.06	1.10	0.96	1.00	cap
SDPSegfault1132	1.47	1.39	1.32	cap	all cap
OptimalAdvertising	0.80	1.11	1.68	3.01	cap
ConvexPlasticity	3.32	5.40	7.31	8.82	cap
UnconstrainedQP	5.73	7.59	11.73	16.78	22.64
`lasso`	0.16	0.15	0.15	0.14	0.15

of its value at the original size on 21 of the 25 problems, at every size where `diffengine` finishes, so the speedup of Table 7 does not come from the particular sizes of the suite. On three of the others, slower problems whose cost grows faster than their data, the ratio grows for the reasons of §3.4: on OptimalAdvertising the number of terms and the number of variables grow together, on ConvexPlasticity the dense block of $\mathbf{P}$ grows with the square of the number of points, and on UnconstrainedQP the dense Jacobian of the chain grows with the size of the Fourier matrix.

`diffengine` exceeds the 20 GB memory limit at five sizes where a `CVXPY` backend finishes. Without the dense blocks, three of them finish, at ratios of 0.31 (CVaR, $s = 2$), 1.57 (ConvexPlasticity, $s = 4$) and 3.18 (OptimalAdvertising, $s = 4$). The other two exceed the limit with and without the dense blocks: SDPSegfault1132 already needs 10 to 13 GB at its original size (Table 9), and at $s = 4$ on Murray, `CPP` exceeds it as well. At the four sizes marked "all cap", every backend exceeds it. The ratio does not depend on the sizes of the

suite, except on three of the slower problems, and the dense blocks cause three of the five sizes at which only `diffengine` runs out of memory.

3.6 Re-solve against a full canonicalization (Q4)

Table 13: Not in the thesis. Re-solve time in milliseconds on the seven DPP problems, mean ± standard deviation over 30 re-solves, each with new parameter values: the cached tensor of `CVXPY`, a full canonicalization on the `ignore_dpp` path, both with the fastest backend, and the cached re-solve of `diffengine`.

	`CVXPY`		`diffengine`
Benchmark	tensor	recompile	cached
ParamSmallMatrixStuffing	1.84 ± 0.01	57.6 ± 0.2	0.438 ± 0.007
ParamConeMatrixStuffing	0.958 ± 0.005	37.7 ± 0.2	0.265 ± 0.004
SimpleFullyParametrizedLP	46.7 ± 0.1	519 ± 1	34.6 ± 0.8
SimpleScalarParametrizedLP	62.8 ± 0.3	1,078 ± 2	70.4 ± 5.9
ParametrizedQP	299 ± 8	2,868 ± 14	349 ± 16
SVMWithL1Regularization	38.3 ± 0.1	2,570 ± 3	125 ± 2
FactorCovarianceModel	20.5 ± 0.1	1,138 ± 3	74.2 ± 0.8

Table 13 compares both caches with what `CVXPY` does without one, canonicalizing again. A re-solve from the cached tensor is 10 to 67 times faster than a full canonicalization, and a re-solve of `diffengine` 8 to 142 times faster. Either cache removes most of the cost of a re-solve, and the gap between the two, at most 3.6 times (Table 5), is small next to it.

3.7 The benchmark problems ((v))

Tables 14 and 15 are Appendix A of the thesis, the description and the dimensions of each problem, for reference in the tables above.

Table 14: Table A.1 of the thesis (p. 70): the 25 problems of the benchmark suite.

Benchmark	Description
SVMWithL1Regularization	Hinge-loss classifier with an ℓ_1 penalty, 25,000 samples and 500 features. The penalty weight is a parameter.
FactorCovarianceModel	Factor-model portfolio selection with 25,000 assets and 250 factors. The risk aversion and the leverage limit are parameters.
Yitzhaki	Gini mean-difference portfolio LP over a dense matrix of all 44,850 pairwise return differences of 50 assets.

Benchmark	Description
SimpleQP	QP with a dense 1,600 × 1,600 quadratic form, 8,000 inequality and 20 equality constraints.
HuberRegression	Huber regression with 3,000 features and 4,500 samples, 12 % of the labels flipped.
LeastSquares	Dense least squares, $\|\mathbf{Ax} - \mathbf{b}\|_2^2$ with $\mathbf{A} \in \mathbb{R}^{7000 \times 2000}$.
SimpleLP	Box-constrained LP with 10^7 variables and a linear objective.
SimpleScalarParametrizedLP	Box-constrained LP with 2×10^6 variables whose cost vector is scaled by a scalar parameter.
SimpleFullyParametrizedLP	Box-constrained LP with 10^6 variables whose cost vector is a parameter.
SlowPruning	Matrix least squares with a 100 × 20 variable and dense data, from `CVXPY` issue 1668.
ParametrizedQP	Box-constrained least squares whose 6,000 × 2,400 matrix and right-hand side are parameters.
Cajas	Gini portfolio LP in ordered-weighted-average form, with 800 vector constraints added one by one.
ConeMatrixStuffing	The LP of SmallMatrixStuffing at size 5,000.
SmallMatrixStuffing	LP with 4,000 variables and 4,000 scalar constraints, each a separate constraint object.
TvInpainting	Total-variation inpainting of a 512 × 512 colour image, with about 30 % of the pixels known.
ParamSmallMatrixStuffing	The LP of SmallMatrixStuffing at size 300, with its matrix, constraint coefficients and right-hand side as parameters.
ParamConeMatrixStuffing	The same parametric LP at size 200.
SemidefiniteProgramming	Standard-form SDP with a 200 × 200 variable and 120 dense equality constraints.
CVaR	Conditional value-at-risk LP over a dense matrix of 131,072 scenarios and 768 variables.
QuantumHilbertMatrix	SDP with partial traces and partial transposes of a 64 × 64 positive semidefinite variable, with sparse constants.
Murray	The Yitzhaki LP written with a dense 244,650 × 700 matrix of pairwise differences.
SDPSegfault1132	Fits a distance matrix through a 99 × 99 positive semidefinite Gram matrix and dense constants, from `CVXPY` issue 1132.
OptimalAdvertising	Allocation of 250 advertisers over 1,000 time slots, with one minimum term per advertiser.
ConvexPlasticity	One step of a convex plasticity model at 3,000 points, with a von Mises yield criterion and three parameters.

UnconstrainedQP	Fits a complex signal through a 252×252 discrete Fourier transform matrix and a Kronecker product with a diagonal of the variable, from `CVXPY` issue 2205. Despite its name, a second-order cone problem.

Table 15: Table A.2 of the thesis (p. 72): dimensions of the problem data of the 25 benchmark problems.

Benchmark	m	n	nnz($\mathbf{A}$)	density (%)	nnz($\mathbf{P}$)	param. size	DPP
SVMWithL1Regularization	51 k	26 k	12.6 M	0.95	—	1	yes
FactorCovarianceModel	75.3 k	75.2 k	6.45 M	0.11	87.5 k	2	yes
Yitzhaki	89.8 k	44.9 k	4.57 M	0.11	—	—	—
SimpleQP	8,020	1,600	12.8 M	99.8	2.56 M	—	—
HuberRegression	13.5 k	16.5 k	13.5 M	6.1	4,500	—	—
LeastSquares	7,000	9,000	14 M	22.2	7,000	—	—
SimpleLP	20 M	10 M	20 M	1.0×10^{-5}	—	—	—
SimpleScalarParametrizedLP	4 M	2 M	4 M	5.0×10^{-5}	—	1	yes
SimpleFullyParametrizedLP	2 M	1 M	2 M	1.0×10^{-4}	—	1 M	yes
SlowPruning	400 k	402 k	8.4 M	5.2×10^{-3}	400 k	—	—
ParametrizedQP	10.8 k	8,400	14.4 M	15.9	6,000	14.4 M	yes
Cajas	641 k	2,450	1.96 M	0.12	—	—	—
ConeMatrixStuffing	5,000	5,000	5,000	0.020	—	—	—
SmallMatrixStuffing	4,000	4,000	4,000	0.025	—	—	—
TvInpainting	2.61 M	1.05 M	3.63 M	1.3×10^{-4}	—	—	—
ParamSmallMatrixStuffing	300	300	300	0.33	—	90.5 k	yes
ParamConeMatrixStuffing	200	200	200	0.50	—	40.3 k	yes
SemidefiniteProgramming	20.2 k	20.1 k	2.43 M	0.60	—	—	—
CVaR	264 k	132 k	25.7 M	0.074	—	—	—
QuantumHilbertMatrix	6,176	6,176	81.1 k	0.21	4,096	—	—
Murray	490 k	245 k	1.5 M	1.2×10^{-3}	—	—	—
SDPSegfault1132	15 k	4,951	1.01 M	1.4	—	—	—
OptimalAdvertising	252 k	250 k	1 M	1.6×10^{-3}	—	—	—
ConvexPlasticity	36 k	33 k	72 k	6.1×10^{-3}	9.05 M	27 k	no
UnconstrainedQP	1,009	289	18.6 k	6.4	—	—	—

References

[1] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.

[2] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[3] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[4] S. Diamond and S. Boyd, "Matrix-free convex optimization modeling," in *Optimization and Its Applications in Control and Data Sciences*, ser. Springer Optimization and Its Applications, B. Goldengorin, Ed. Springer, 2016, vol. 115, pp. 221–264.

[5] D. Cederberg, W. Zhang, P. Nobel, and S. Boyd, "Disciplined nonlinear programming," arXiv:2606.02896, 2026.